Atal Bihari Vajpayee
Indian Institute of Information Technology
and Management, Gwalior



Auditable IoT Energy Data Pipeline

Project report
IoT Big Data Management

Submitted by

Nishant Harkut
Enrollment number: 2023IMG-040

Submitted to

Dr. Amandeep Kaur
Instructor

13 November 2026

Project repository

`https://github.com/nishantharkut-research/IoT-Energy-Data-Pipeline`

Abstract

Smart-meter data can reach an analytics platform through repeated, delayed or malformed deliveries. A successful processing job alone gives little information about whether these deliveries produced a complete, usable result. This project builds an auditable energy-data pipeline that preserves received deliveries, reconstructs exact source records and checks the finalized result through a second implementation.

Public HKUST smart-meter workbooks and Brick metadata are registered and converted into exact canonical records. Deterministic Kafka replay introduces controlled delivery faults. Spark stores deliveries in Bronze, maintains an event-time-bounded Silver view and rebuilds finalized Gold from Bronze. Independent Hadoop Streaming programs compute reference records and aggregates from canonical NDJSON. Delivery accounting, replay termination and exact comparisons must agree before publication.

Fifteen runs were completed: nine correctness, recovery and watermark fixtures, plus three repetitions each at 100K and 1M source records. All declared Spark/Hadoop comparisons agree. A restrictive watermark leaves 143 records in Silver, while Gold retains all 144 source records. Interrupted and uninterrupted fixture executions reproduce four semantic outputs. Median verification time is 13.88 minutes at 100K and 165.30 minutes at 1M.

A read-only dashboard presents source quality, delivery dispositions, verification, recovery and storage/query costs. Consumption analysis distinguishes matching intervals from unsupported comparisons: the measured discrepancy is zero on 138 comparable periods, while two unmatched periods leave total exposure unknown. The proposed Indian application is a DISCOM review of metering-provider data, evaluated here using historical HKUST replay rather than a live utility deployment.

Keywords: smart-meter data, delivery accounting, Apache Kafka, Apache Spark, Hadoop Streaming, independent verification.

Contents

1 Introduction **4**

2 System architecture and technology choices **6**

3 Dataset and source preparation **7**

4 Delivery processing and finalization **9**

5 Independent Hadoop verification **11**

6 Experiment design and controlled-fault results **12**

7 Watermark behaviour and consumption interpretation **13**

8 Recovery and reproducibility of the finalized answer **15**

9 Scale, storage and SQL measurements **16**

10 Dashboard implementation and utility-review workflow **17**

11 Discussion and conclusion **20**

Reading the report. Sections 1–3 explain the problem, architecture and data. Sections 4–5 describe processing and verification. Sections 6–9 present measured results. Section 10 explains the dashboard and proposed utility-review workflow. Figures are placed with their discussion. Full desktop captures are also supplied with the project.

Dashboard views in this report

View	Page
Overview and experiment register	5
Source audit	8
Delivery inspector	11
Fault comparison	14
Independent verification	12
Watermark and consumption review	15
Recovery	16
Storage and SQL	18
Indian utility-review workflow	19

1 Introduction

1.1 Problem and motivation

Smart-meter analytics depends on more than the value recorded by a meter. The platform must also establish which readings arrived, which were rejected, which were repeated and which contributed to the final result. A retry creates another delivery of the same source event. A delayed reading may arrive after an operational stream has released its state. A malformed message must remain traceable even when it is excluded from analysis.

The project addresses the following problem: *how can a completed meter-data result be checked against its source and delivery history, rather than accepted because a processing job finished?* The solution preserves received evidence, accounts for delivery outcomes, reconstructs exact records after replay completion and compares them with a separately implemented computation.

1.2 Connection to Indian smart-meter systems

The intended user is a distribution-company (DISCOM) analyst reviewing outputs from an advanced metering infrastructure service provider (AMISP). A head-end system (HES) collects readings, and meter-data management (MDM) systems apply validation, estimation and editing (VEE). The analyst needs to distinguish source-data gaps from delivery exceptions and processing differences before interpreting a reported energy result.

A CSIS survey published on 30 October 2026 reports communication and data-integration limitations in TNPDCL and Tata Power-operated Odisha utilities [10]. A CAG report also records historical communication and reporting problems in a Tripura smart-grid project during 2018–2021 [11]. These are specific examples motivating data review, not estimates of national failure prevalence.

Raw-data retention, VEE and quality monitoring already exist. MSEDCL's 2022 tender requires original received data, VEE data and edit trails [7], while commercial MDM systems expose quality and timeliness metrics [8]. Quality control for AMI data is also established in the literature [9]. This project contributes an evaluated, reproducible integration of delivery accounting, exact reconstruction, independent computation and explicit comparison coverage. It is intended to support review alongside existing systems.

1.3 Objectives and completed scope

The implementation has five objectives:

1. Preserve and account for each received delivery, including retries and rejected messages.
2. Separate the bounded operational result from the exact finalized source result.
3. Verify source records and aggregates through independent Hadoop programs.
4. Demonstrate recovery without changing the declared semantic outputs.
5. Present quality, consumption support and measured processing costs in a usable review interface.

The evaluation contains 15 completed runs through 1M selected source records. The 5M experiments were excluded because of resource and time constraints. The Indian application is a proposed workflow, supported by public-source motivation and tested mechanisms. Field billing savings and live utility reliability are outside the evaluation.

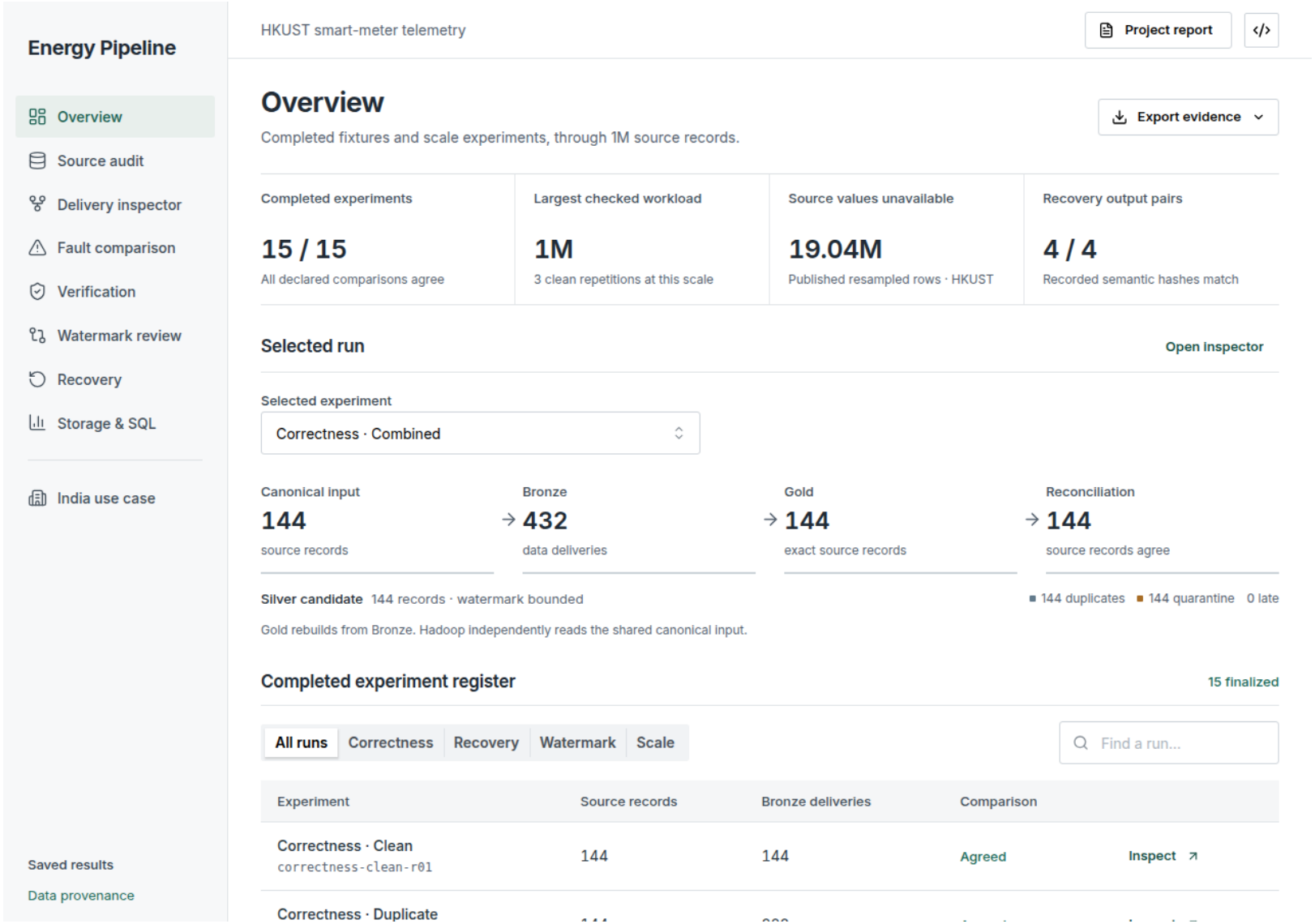


Figure 1: Experiment register and selected-run accounting in the desktop application.

2 System architecture and technology choices

Figure 2 shows the two computation paths. Source registration and canonicalization are shared. Kafka and Spark form the delivery path. Canonical NDJSON also enters HDFS for Hadoop Streaming. Both outputs are checked before finalization. The comparison is independent in implementation after canonicalization, not independent in physical measurement or source preparation.

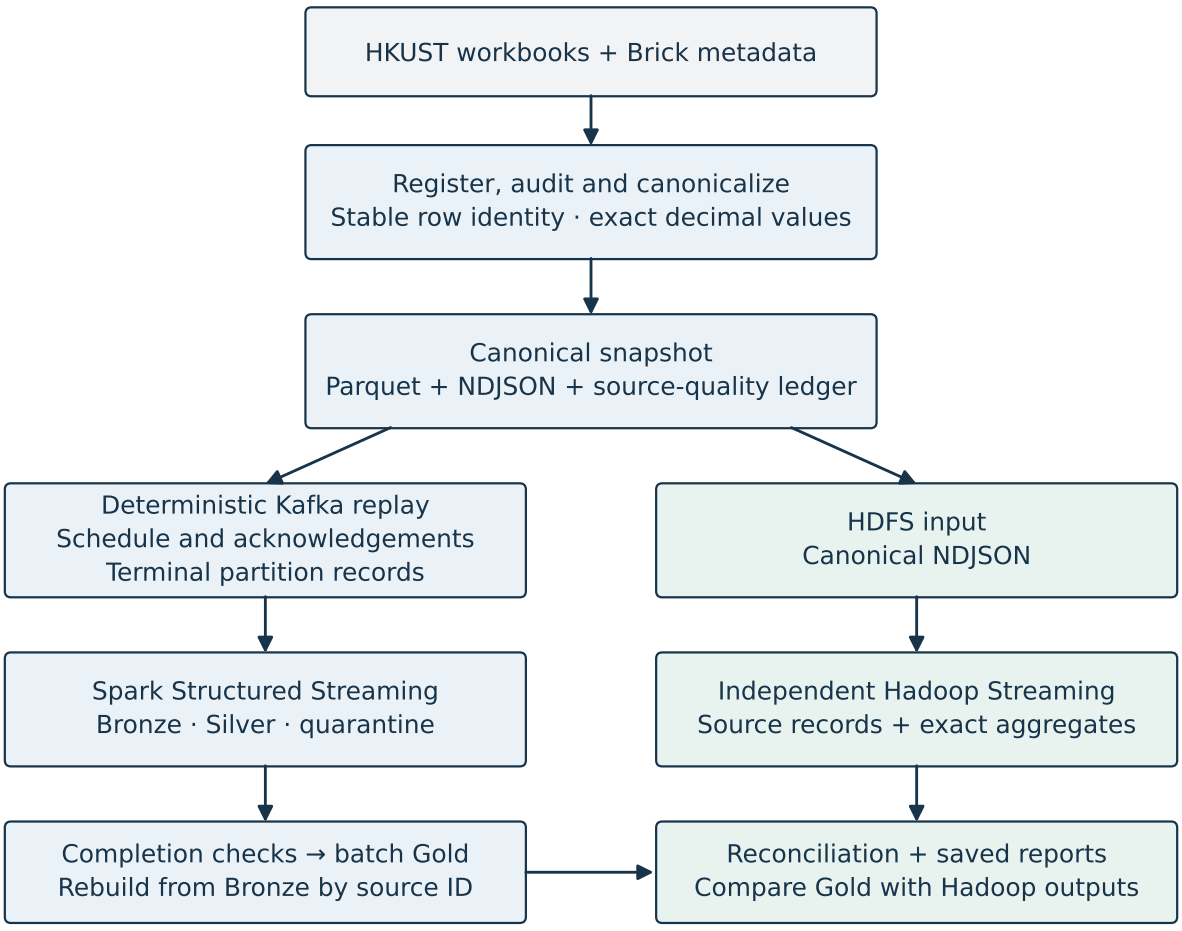


Figure 2: Processing path and independent reference computation share a declared canonical input.

Table 1: Technology choices and their role in the implementation.

Component	Role and reason for use
Kafka 3.7.1	Partitioned delivery log, acknowledgement tracking and reproducible replay [5].
Spark 3.5.1	Stateful event-time processing and batch reconstruction from retained deliveries [4].
Hadoop 3.4.3	HDFS input and standalone Streaming programs for a separately implemented reference [6].
NDJSON / Parquet	Explicit record interchange for verification / compressed columnar storage for analysis.
SQLite	Disk-backed ordering and interval processing with bounded working memory.
React / TypeScript	Read-only desktop review of saved results, with Mantine components and Recharts plots.
JSON manifests	Bind source, configuration, completed outputs and presentation inputs by recorded hashes.

2.1 Review workflow and storage decisions

The reviewer begins with the source population, selects a completed experiment, checks delivery conservation and exact comparisons, then interprets consumption coverage and execution cost. Figure 1 shows the application entry point. Its experiment register links these review tasks to the same selected run.

The workload is historical replay and read-only analysis. Files provide the durable experiment outputs, while SQLite supports local ordered processing. A separate serving database such as HBase would add operational complexity without answering a measured random-access requirement in this project. Parquet and Spark SQL provide the implemented analytical path.

Docker Compose provides Kafka, Spark and Hadoop services for the original experiments. The portable dashboard uses React and TypeScript, Mantine interface components, Recharts plots and Lucide icons to present saved summaries. A Python export step preserves the exact evidence bytes; the browser checks their SHA-256 bindings. The original Streamlit sealed-run viewer remains available for inspecting original local artifacts. The portable viewer runs separately from the experiment services and uses committed result summaries.

3 Dataset and source preparation

3.1 Public telemetry and metadata

The source is the HKUST campus smart-meter collection deposited on Dryad under DOI `10.5061/dryad.k3j9kd5h6`, with a companion data descriptor [1, 2]. The project registered the deposited version-5 files. Its counts describe those files, rather than substituting headline meter totals from the article. The package contains raw and publisher-resampled variants. Brick metadata provides semantic relationships and recorded units [3]. Raw and resampled records remain separate throughout preparation.

The published source audit records 1,394 raw workbooks and 1,329 resampled workbooks. All workbook meters in each variant join to Brick. The graph contains 1,432 meter references. Its additional references do not establish missing telemetry for a particular analysis window. The 65 raw workbooks absent from the resampled set have no physical data rows in the audit. This observation does not establish the authors' reason for excluding them.

Table 2: Published source-audit counts. Categories can overlap.

Observation	Raw	Resampled
Workbooks	1,394	1,329
Physical data rows	50,903,710	63,273,828
Finite numeric readings	50,903,710	44,238,638
Missing numeric cells	0	19,035,190
Negative readings	134,612	134,612
Ordered numeric decreases	50,675	67,878
Duplicate decoded timestamps	853,595	0
Out-of-order timestamps	147	0

3.2 What availability means

For the published resampled grid, the numeric-cell availability is

$$A_{\text{numeric}} = \frac{44{,}238{,}638}{63{,}273{,}828} \times 100\% = 69.916\%. \tag{1}$$

The denominator is the number of published rows. It is not an independently expected reading schedule. In particular, zero missing numeric cells in raw workbooks does not prove continuous temporal coverage. Negative

values and ordered decreases are review observations, not confirmed meter failures or resets. Figure 3 presents these distinctions in the portable dashboard.

3.3 Registration and preservation

Registration records relative paths, byte sizes and SHA-256 hashes. Structural profiling examines workbook sheets and cells. The exhaustive telemetry audit records value, timestamp and ordering observations. Numeric canonical events and source-quality exclusions are retained separately. Source-quality exclusions are not Kafka quarantine: the former concern preparation, while the latter concerns delivered messages.

The prepared snapshot accounts for 63,273,828 source rows as 44,238,638 numeric records and 19,035,190 source-quality records. This accounting includes finite numeric values that may still have quality flags. It does not declare all numeric readings physically valid.

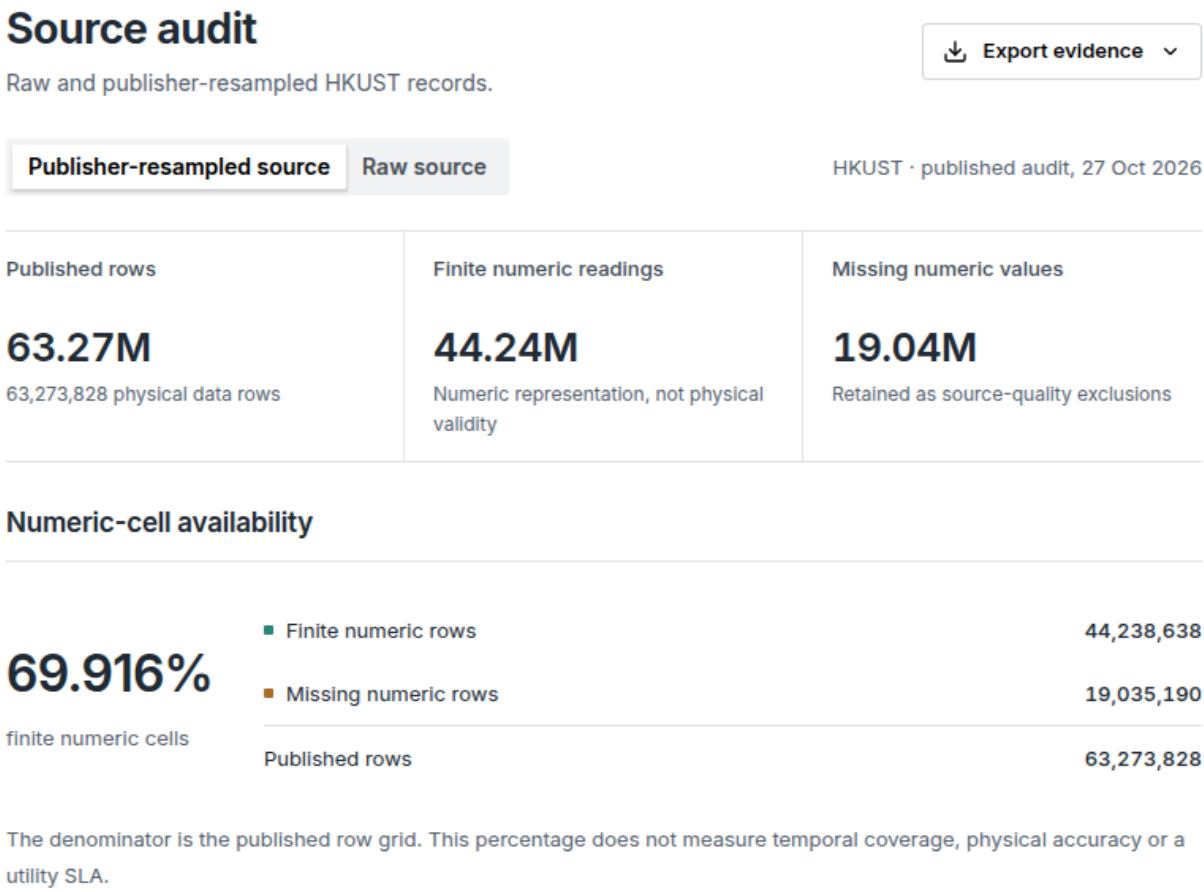


Figure 3: Raw/resampled selection, numeric-cell availability and source-registration counts.

3.4 Canonical identity and exact representation

3.4.1 Source rows and delivery identities

The source identifier is a SHA-256 digest of a structural locator:

$$\text{source_event_id} = H(v \parallel s \parallel p \parallel w \parallel r), \tag{2}$$

where v is dataset version, s is source variant, p is relative path, w is worksheet and r is physical row. The implementation serializes these fields with a declared separator and rejects ambiguous components. Identity

does not depend on a value, timestamp alone, or a model-generated key. A repeated Kafka delivery therefore retains the same source identity but has its own delivery identity.

This choice matters when two physical rows share a timestamp. They remain distinct source events. Gold deduplicates repeated deliveries by exact source identity; it does not erase different physical rows merely because their measurement keys match. Consumption derivation applies a separate timestamp policy after source reconstruction.

Table 3: Selected canonical fields and their purpose.

Field group	Purpose
Dataset version, variant, path, sheet, row	Recover the original physical source location.
Source event identifier	Deduplicate repeated deliveries without merging distinct physical rows.
Original timestamp and numeric text	Retain stored representations for interpretation and exact conversion.
Event time and source timezone	Make the chosen timezone interpretation explicit.
Scaled value and decimal scale	Preserve numeric values with exact decimal arithmetic.
Unit and measurement kind	Restrict valid consumption derivation.
Brick identifiers and quality flags	Retain semantic relationships and observed limitations.

3.4.2 Numeric precision and time interpretation

For a stored decimal x with scale d, the canonical value is the exact integer $z = x10^d$. The scale accompanies the value. The source reader retains stored cell text before conversion through a binary floating-point representation can change it. A bounded check found that meter D0779's stored value `26724.7`, at scale 15, requires integer `26724700000000000000`. It exceeds signed 64-bit range. The implemented storage contract uses signed values within a 38-digit decimal envelope, checked through the Spark representation path. This is a representation finding, not evidence of an incorrect physical reading.

All audited timestamps are naive. The executed snapshot interprets them in `Asia/Hong_Kong` under an explicit experimental assumption. The dataset authors did not confirm that convention. Original timestamp text is preserved, and absolute UTC boundaries remain conditional on the assumption. The report does not reinterpret HKUST observations as Indian household measurements.

3.4.3 Bounded-memory preparation

The resumable preparation path creates sorted workbook units, writes checkpoints, and merges bounded groups into canonical NDJSON and the quality ledger. It subsequently writes Parquet and the snapshot manifest. The per-workbook checkpoints support preparation recovery without keeping the complete source in memory. They are intermediate work products, distinct from the immutable completed snapshot. The full source is prepared once; the scale runs select source-identifier-sorted prefixes from the numeric snapshot. Those prefixes are not guaranteed continuous meter-days and cannot support an outage-duration or time-of-use study.

4 Delivery processing and finalization

4.1 Replay records and terminal evidence

A deterministic replay schedule fixes the seed, source selections, partition assignments and injection types. Delivery records retain source identity and delivery identity. Producer receipts record acknowledgement information. Terminal partition records establish that the planned replay has ended for each partition. A quiet Kafka topic alone is insufficient proof of completion.

Bronze archives data deliveries with their transport evidence. Terminal controls are retained and checked separately. Parsing and contract validation then distinguish valid source events from malformed or contract-violating messages. Quarantine preserves rejected payloads with categorized reasons so that exclusion does not erase delivery evidence.

4.2 Operational Silver and finalized Gold

Silver is an operational candidate. Its event-time watermark bounds state and the period over which duplicate suppression can be maintained. Late valid deliveries are accounted for separately from invalid deliveries. Spark's documented watermark and deduplication semantics motivate this distinction [4]. Silver can therefore be useful while still being unsuitable as the sole source for an exact completed result.

Gold is a batch reconstruction from Bronze after completion checks. It parses valid deliveries, deduplicates exact source identities, and computes finalized source records and aggregates. This path retains received valid records that Silver omitted because of the operational watermark. It cannot recover a physical reading that was never supplied to the pipeline. Figure 4 shows the control and data paths.

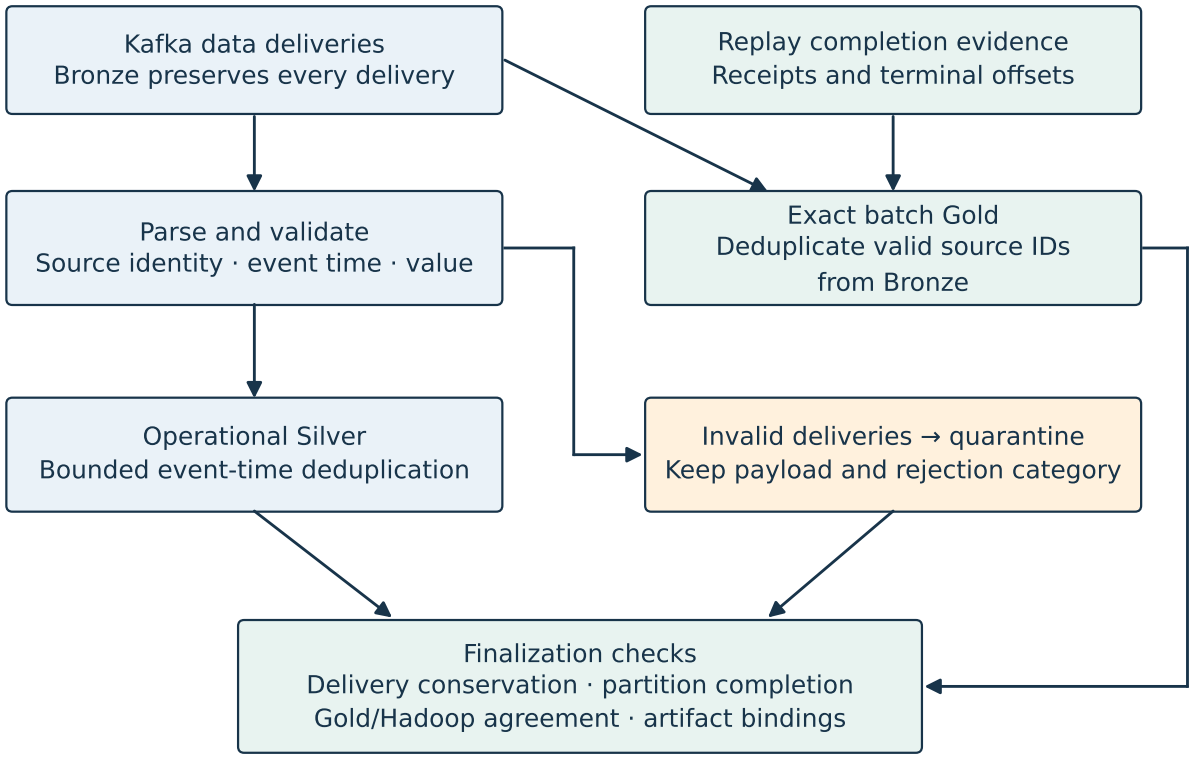


Figure 4: Delivery dispositions and exact finalization. Gold uses Bronze rather than inheriting Silver's omissions.

Figure 5 presents the selected run and its delivery accounting. The layer audit checks two conservation equations. Let B be Bronze data deliveries, V parsed-valid deliveries, Q quarantine deliveries, S Silver records, D duplicate-valid dispositions and L late-valid dispositions. Under the recorded run definitions,

$$B = V + Q, \tag{3}$$

$$V = S + D + L. \tag{4}$$

The completion check separately binds acknowledged data deliveries to Bronze and verifies terminal partition coverage. The exact Gold count is a source-event count, not another delivery-disposition category.

4.3 Fail-closed publication of completed runs

Finalization requires the replay manifests, delivery totals, terminal offsets, layer dispositions and exact comparisons to reconcile. Decision, query and storage reports must bind to the corresponding finalized artifacts. A run manifest registers the artifacts and is sealed with a SHA-256 binding. The original sealed-run dashboard checks both this manifest and its registered files and rejects incomplete, altered or unreconciled runs. These checks establish byte bindings and declared processing consistency, not adversarial authenticity or physical accuracy.

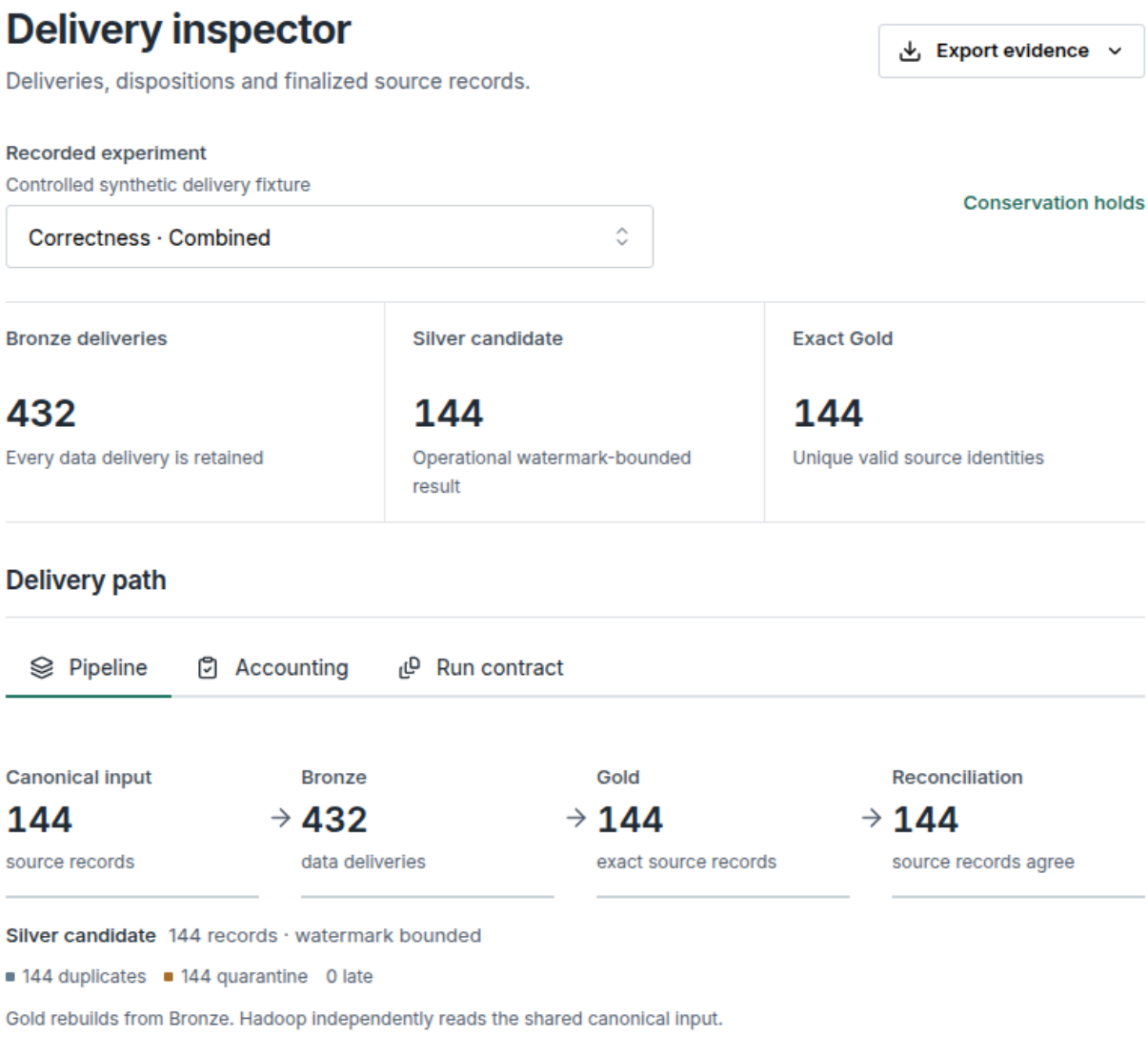


Figure 5: Selected-run delivery accounting and processing layers.

5 Independent Hadoop verification

The Hadoop path reads canonical NDJSON rather than Spark's Gold result. Its standalone source-record program emits the declared exact representation, while the mapper/reducer path produces grouped counts, exact scaled-value sums, timestamp bounds and quality counts. Python integer arithmetic supports exact checksum accumulation within the declared contract. Group dimensions include the meter, unit, measurement kind and decimal scale, avoiding addition of incompatible representations.

Reconciliation sorts and compares exported source records by their declared identities and fields. It also compares aggregate records field by field. The source comparison covers 23 exact fields, including source location, original text, event time, scale, value, units, quality flags and Brick relationships. The aggregate

comparison covers 10 fields. This makes the comparison contract inspectable rather than reducing it to an unqualified “passed” label.

Checksums of cumulative readings in aggregate records are useful exact processing checks. They are not energy-consumption totals. Consumption is computed separately from valid differences between supported successive readings. This prevents interpreting a large sum of meter-register values as electricity delivered during a period.

Figure 6 exposes the exact-field contract and selected-run comparison. All 15 saved comparisons have matching source/aggregate counts and empty error lists. The largest checked source population is 1M selected canonical records. Independence ends at the shared canonical input boundary: both computations can agree on an incorrectly normalized input. The design therefore retains source-audit evidence and the assumptions used during preparation instead of treating cross-engine agreement as physical truth.

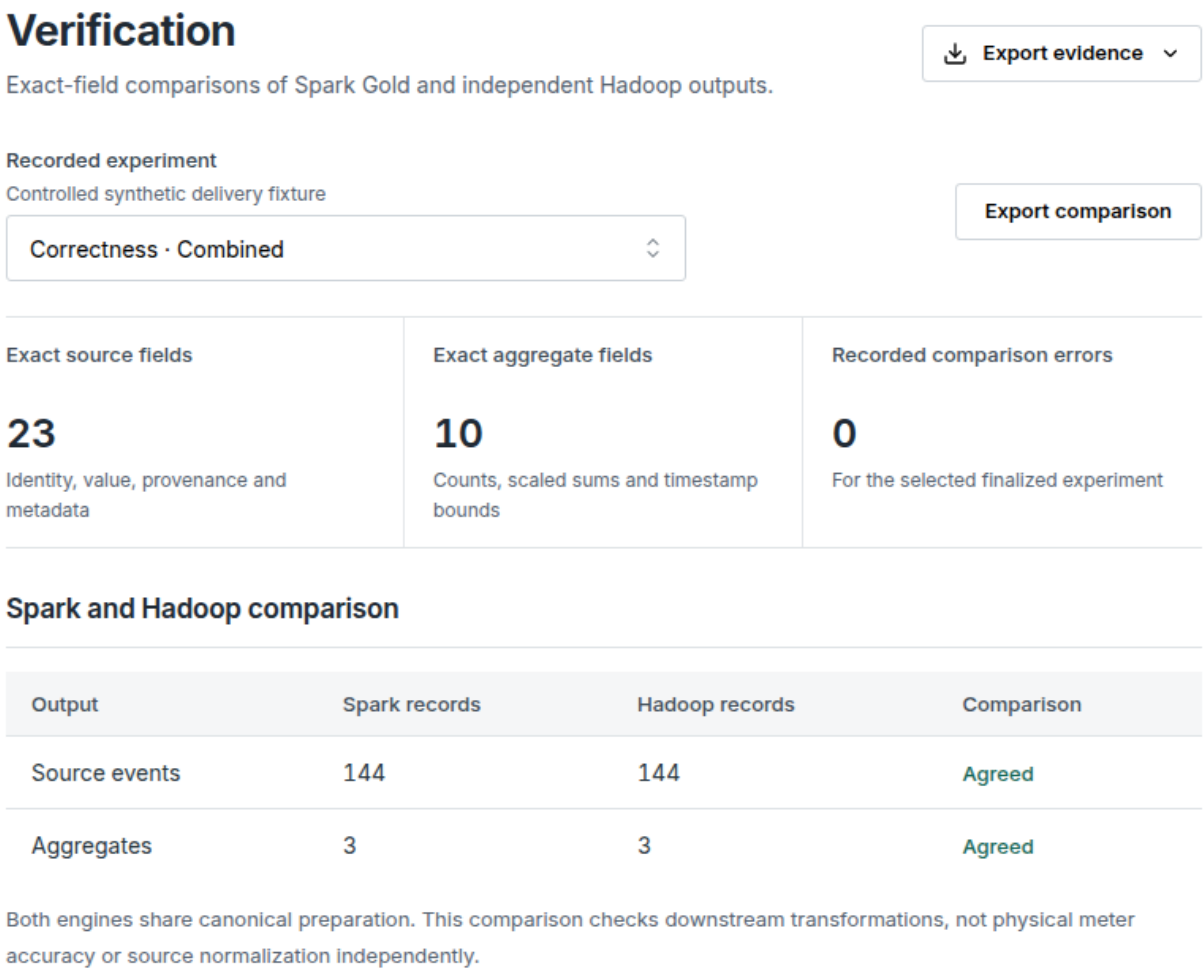


Figure 6: Recorded Spark/Hadoop source and aggregate comparisons with the exact-field contract.

6 Experiment design and controlled-fault results

The final matrix contains five correctness schedules, two recovery executions, two watermark configurations and six real-data scale executions. Each correctness schedule uses a 144-source-event fixture. The clean scale workloads use 100K or 1M selected canonical numeric records, with three repetitions at each scale. The originally planned 5M repetitions are explicitly excluded for resource and time constraints.

Table 4: Executed streaming configurations. All use the declared stack in Table 1.

Family	**Partitions**	**Offsets/trigger**	**Watermark**
Correctness / recovery	3	60	24 hours
Watermark comparison	1	1	24 hours / 1 minute
Clean scale	3	10,000	24 hours

Table 5: Fixture layer counts from the saved closure report. Terminal controls are separate.

Fixture	Bronze	Silver	Dup.	Late	Quar.	Gold
clean	144	144	0	0	0	144
duplicate	288	144	144	0	0	144
delayed	144	144	0	0	0	144
malformed	288	144	0	0	144	144
combined	432	144	144	0	144	144
recovery-uninterrupted	144	144	0	0	0	144
recovery-interrupted_resumed	144	144	0	0	0	144
watermark-retaining	144	144	0	0	0	144
watermark-restrictive	144	143	0	1	0	144

Figure 7 compares the five delivery schedules. The duplicate schedule repeats unchanged valid deliveries. The malformed schedule adds rejected messages alongside valid originals. The combined schedule includes both effects. Consequently, these schedules test delivery preservation and disposition under declared faults. They do not simulate arbitrary physical-meter corruption or guarantee a nonzero consumption error.

In the combined fixture, 432 Bronze deliveries split into 288 parsed-valid and 144 quarantined deliveries. The valid deliveries split into 144 Silver records and 144 duplicate dispositions. Gold contains the same 144 source records as the clean execution. The five correctness schedules have identical hashes for each of the four corresponding semantic outputs: Gold source records, Gold aggregates, Hadoop source records and Hadoop aggregates. This is stronger than merely reporting equal row totals.

The source and aggregate error lists are empty for all 15 completed runs. These results support the implemented fault-accounting and reconstruction behaviour under the selected schedules. They do not estimate the frequency of such faults in HKUST infrastructure or Indian AMI deployments.

7 Watermark behaviour and consumption interpretation

The retaining watermark fixture preserves 144 source records in Silver. The restrictive configuration preserves 143, with the remaining valid record classified as late. Both executions produce identical finalized Gold and Hadoop source records. Thus the observed difference is an operational candidate omission, not a failure of exact Gold reconstruction.

Consumption is derived only for records whose declared semantics support cumulative kWh. For two eligible readings, interval energy is

$$\Delta E_{m,i} = E_m(t_i) - E_m(t_{i-1}). \tag{5}$$

The derivation requires a valid prior baseline and unambiguous timestamps. Negative differences and unresolved resets are flagged. The conservative duplicate-timestamp policy also excludes affected derivations and the first subsequent interval requiring a new baseline.

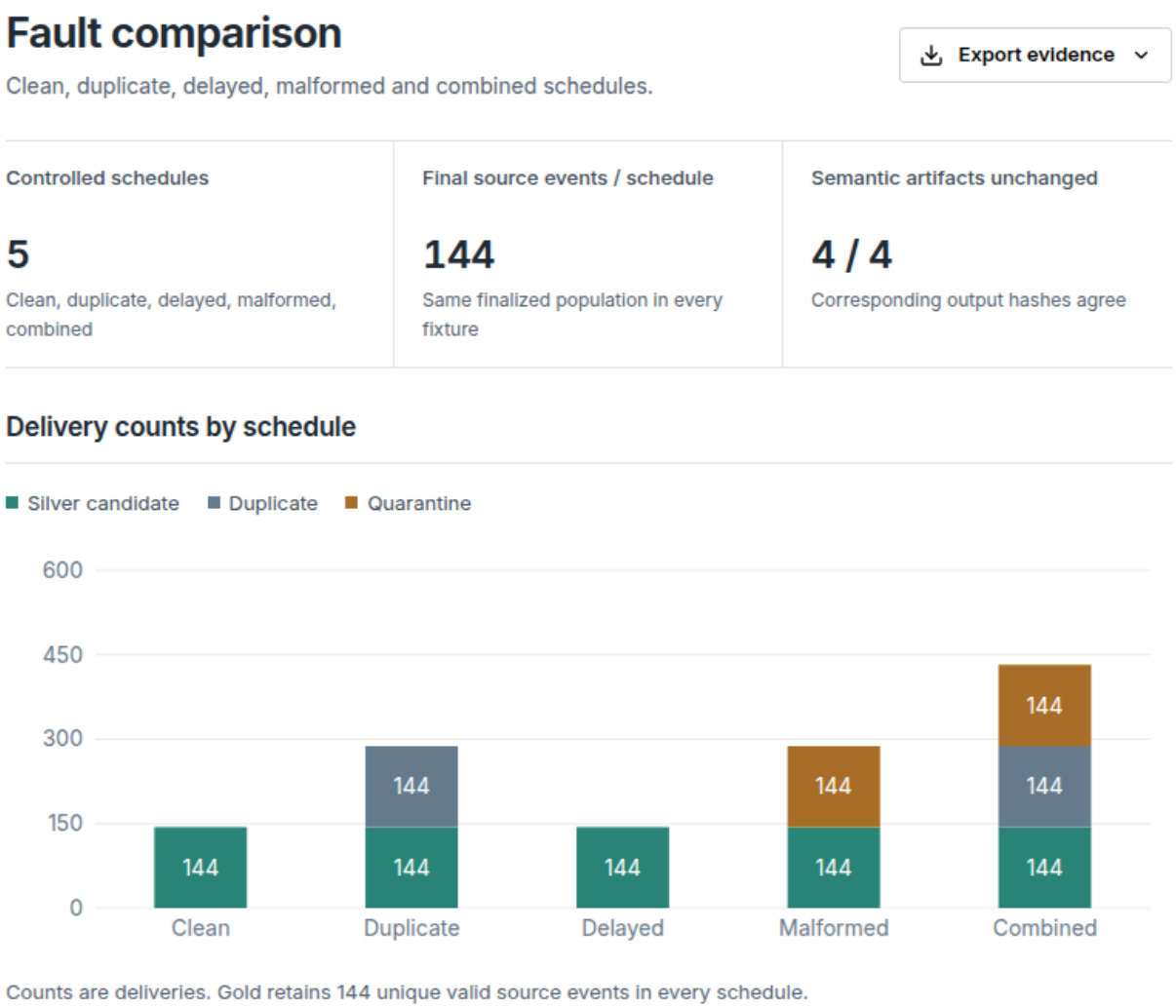


Figure 7: Delivery dispositions across the five controlled schedules.

A numeric comparison requires the same meter, start time and end time in both series. Let C denote that comparable-support set. The recorded discrepancy is

$$D_C = \sum_{(m,a,b)\in C} \left| \Delta E^{\text{Silver}}_{m,a,b} - \Delta E^{\text{Gold}}_{m,a,b} \right|. \tag{6}$$

It is a subtotal over C, not an error estimate for intervals outside it.

Table 6: Consumption support in the restrictive-watermark execution.

Observation	Recorded result
Candidate / verified intervals	140 / 138
Comparable / incomparable periods	138 / 2
Candidate / verified flagged records	3 / 6
Absolute discrepancy on comparable support	0 kWh
Total residual exposure	Unknown

Table 6 separates a numeric subtotal from its coverage. The two unmatched periods have no supported numeric error estimate. In particular, they do not represent a measured 2 kWh loss. The dashboard presents the omission and the comparison support together (Figure 8).

Scenario rates of 0.5, 1.0 and 2.0 currency units per kWh are declared hypothetical rates. They are neither HKUST tariffs nor Indian billing rates. A zero subtotal on comparable intervals yields a zero scenario subtotal there, while total energy error and residual financial exposure remain unknown. The project demonstrates preservation, reconstruction and careful interpretation of coverage; it does not demonstrate actual financial savings.

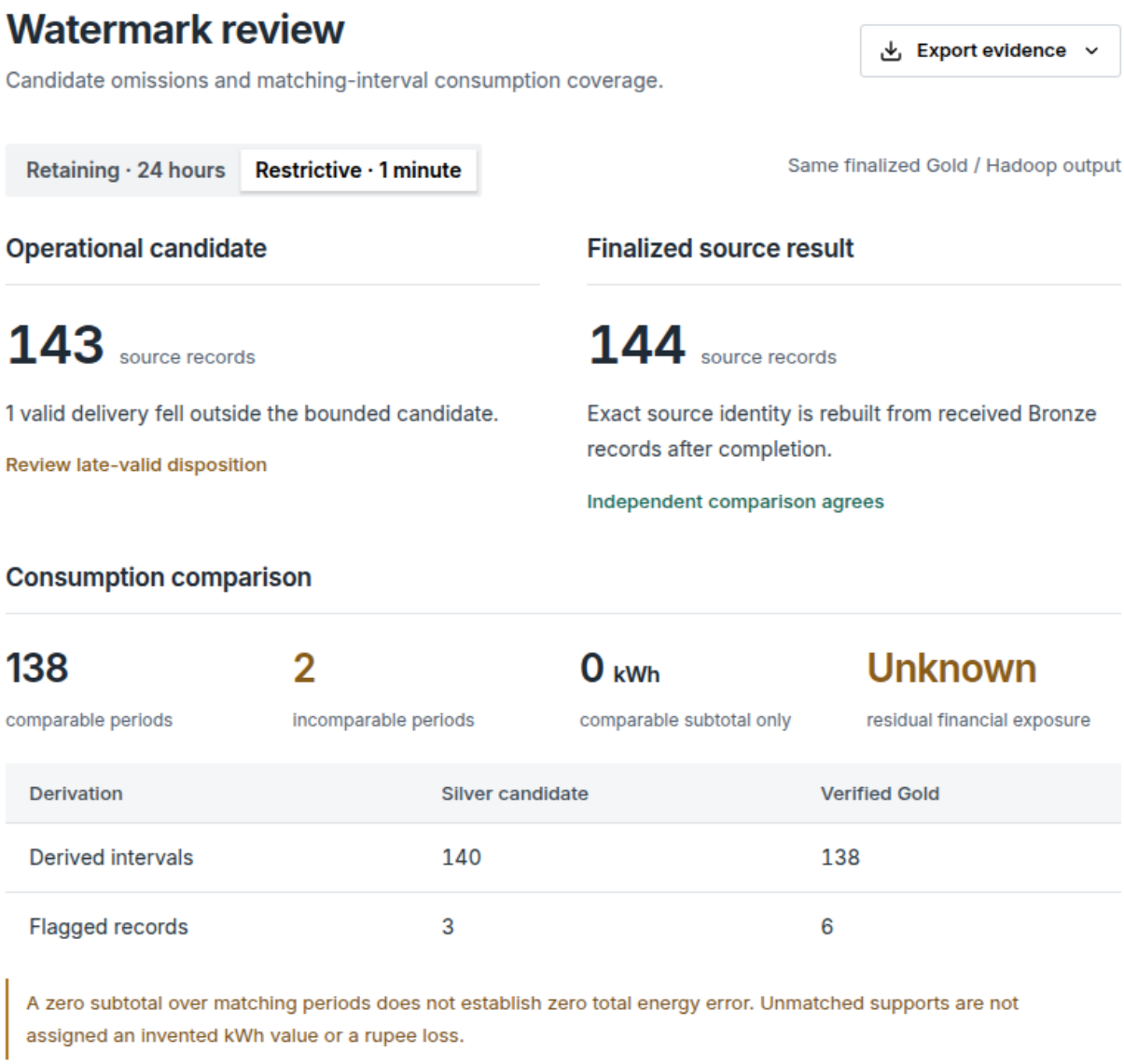


Figure 8: Restrictive Silver omission, exact Gold and consumption-comparison coverage.

8 Recovery and reproducibility of the finalized answer

Figure 9 presents the saved output comparisons. The recovery fixture compares uninterrupted execution with an interrupted-and-resumed execution. For each semantic artifact, its full SHA-256 digest is compared across the two executions. The four pairs agree. Transport offsets, timing files and physical Parquet layouts are not required to match, because they describe execution details rather than the declared semantic output.

The scale executions also required recovery. Saved receipts document resumed stages and configuration bindings. All three 1M runs finalized, but their complete workflow durations are unavailable. The report preserves that distinction. A recovered result can still support the recorded exact comparison and the measured component costs without supplying an uninterrupted end-to-end latency measurement.

The portable submission checks committed summary bytes against an evidence index, verifies original plan/scope bindings, checks the 15 run identities and recomputes delivery-conservation equalities. Original runtime verification was recorded on 2 November 2026. The portable report does not repeat that per-artifact audit of the large original directories. The strict sealed-run viewer remains available when those directories are supplied.

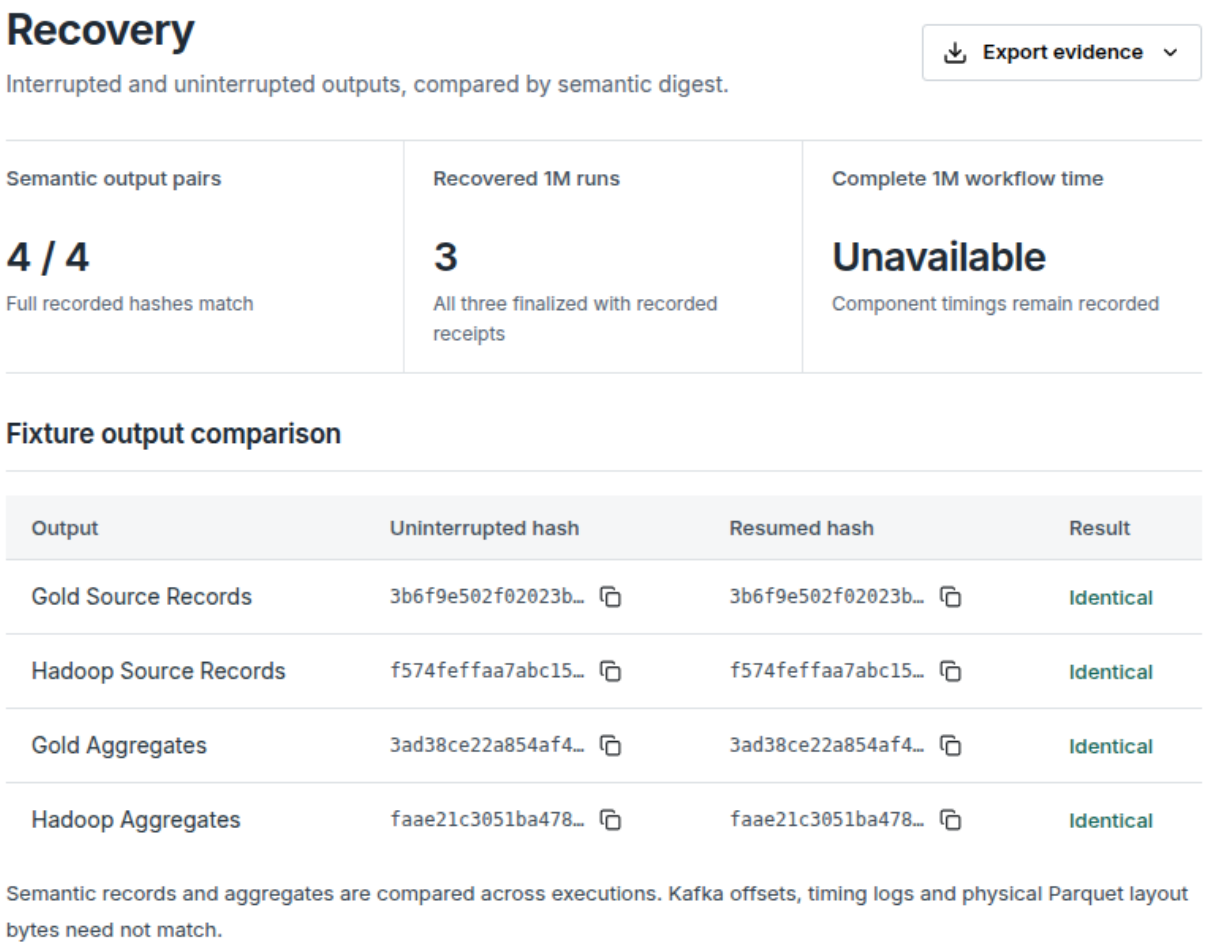


Figure 9: Matching fixture output digests and saved scale-recovery receipts.

9 Scale, storage and SQL measurements

9.1 Verification cost

Verification time includes Gold finalization, layer auditing, Hadoop computation and reconciliation. It excludes the streaming stage and subsequent storage/query benchmarking. Verification storage covers the declared finalized verification artifacts, not every checkpoint, log or service volume on disk. Table 7 reports the three repetitions at each scale. Original CPU and memory specifications are absent from the portable closure, although

Table 7: Recorded verification costs. Time is minutes; storage is decimal MB.

Events	Minimum	Median	Maximum	Median storage
100,000	13.63	13.88	14.10	252.78
1,000,000	163.42	165.30	165.97	2522.71

the execution plan binds the original machine metadata. These are single-host measurements. The 1M median

is about 11.91 times the 100K median for a tenfold event-count increase. This is a descriptive ratio on one machine, not a claim of asymptotic complexity or production throughput. The absolute cost matters: full independent recomputation is substantially more expensive than showing a job-completion status. The current project measures that cost rather than claiming it negligible.

9.2 Storage formats

Figure 10 provides the selected-run storage and cost view. Table 8 compares the finalized NDJSON export with Zstandard Parquet outputs. Ratios are stored bytes divided by NDJSON bytes. The same ratio is recorded in all three repetitions at each scale. Unpartitioned compressed Parquet is smaller in these workloads. Partitioning adds file and metadata overhead; no benchmark here establishes that partitioning improves query speed.

Table 8: Recorded storage sizes and ratios for each scale.

Events	NDJSON (MB)	Parquet/NDJSON	Partitioned/NDJSON
100,000	112.04	0.0615	0.3038
1,000,000	1120.65	0.0601	0.1654

9.3 Query workloads and timing limits

Five read-only SQL workloads cover meter, building, time window, quality and exposure. Each runs three timing repetitions and records a complete ordered-result digest. The query policy uses cached Gold input and consumes projected JSON results, including serialization, transfer and hashing. These are complete cached-result workloads, not isolated SQL-operator latencies or cold-storage read measurements.

The 100K runs use the original settings. Smaller query partitions were used at 1M. Their timings therefore cannot be treated as a configuration-controlled scaling benchmark. A zero-row quality query in a selected numeric prefix also does not contradict full-source quality findings. Different populations and definitions require separate denominators. Per-run minimum, median and maximum timings and result digests remain available in the committed results instead of being replaced by a single favourable run.

10 Dashboard implementation and utility-review workflow

10.1 From experiment results to a review interface

The dashboard has nine task-based views: overview, source audit, delivery inspector, fault comparison, verification, watermark review, recovery, storage/SQL and India use case. The interface uses a consistent light theme, readable metric rows, local table scrolling and visible units. The overview establishes the experiment context. The remaining views answer one review question at a time.

Selecting a run updates its recorded counts and comparison contract. Search and filtering operate on the experiment register. The source selector switches between raw and resampled populations. The watermark selector exposes the retaining/restrictive result. Run dialogs, CSV exports, JSON downloads and the report link provide access to the underlying saved information. The viewer starts no experiment services and presents no simulated live meter feed.

A Python exporter copies the bounded evidence files without changing their bytes. The browser checks their SHA-256 bindings before displaying results. Missing or altered inputs are rejected. The separate Streamlit sealed-run viewer checks original artifact directories, including their registered files, when those are available. This distinction makes the submission reviewable without requiring the external SSD while retaining the original artifact-verification path.

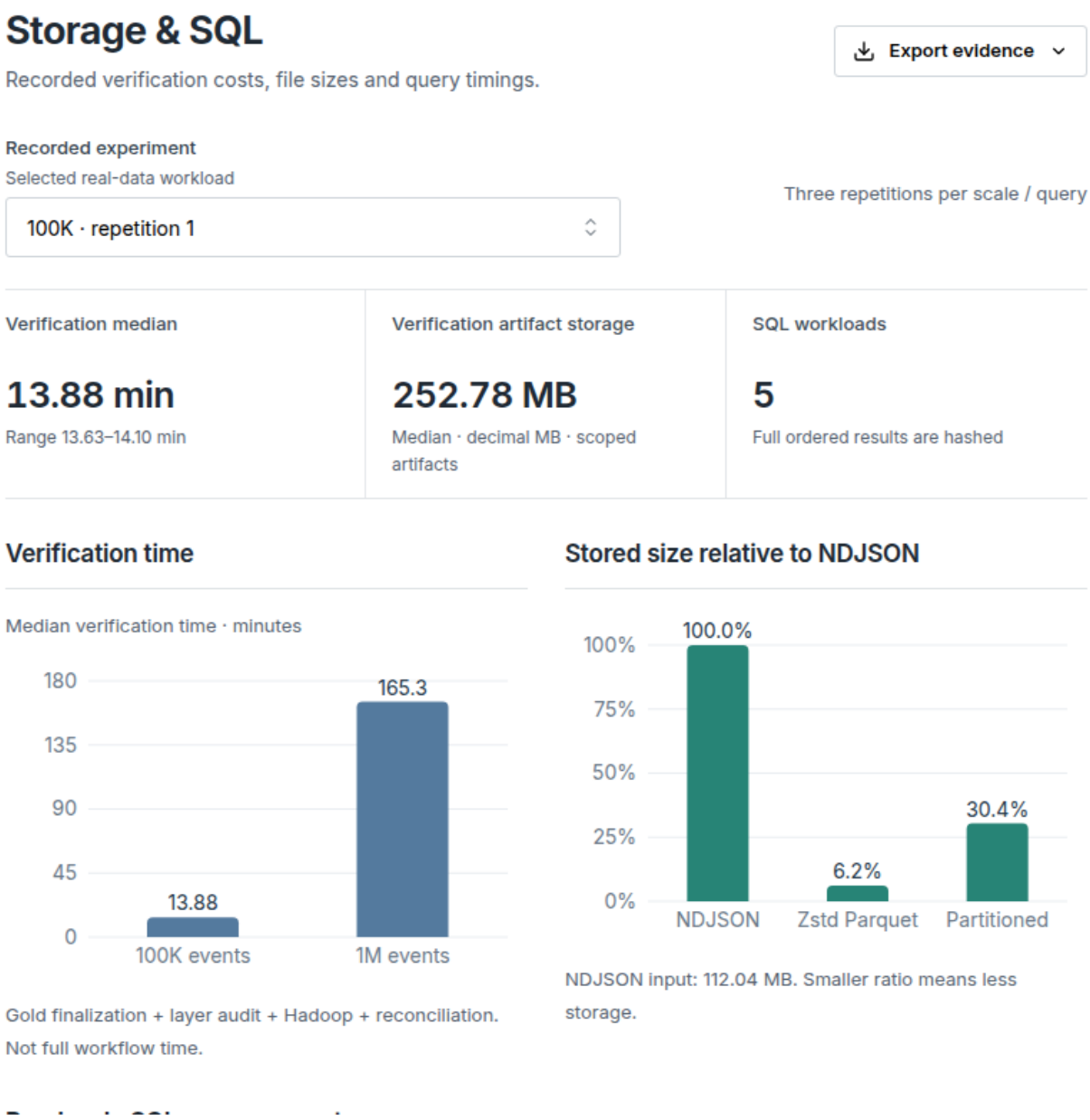


Figure 10: Verification cost and storage results for the selected repetition.

10.2 Proposed Indian utility application

Figure 11 maps the implemented mechanisms to a DISCOM analyst's review. The first step is to identify the source population and its missingness. The next is to account for deliveries and inspect processing exceptions. A finalized result is compared against an independent implementation before consumption coverage is interpreted. This is an accountability workflow alongside HES/MDM/VEE, rather than a replacement for them.

Table 9: Review questions and the evidence demonstrated in this project.

Review question	Demonstrated evidence
Was the supplied source population complete?	Source audit distinguishes missing published cells from delivered-message quarantine.
Did delivery behaviour affect the operational view?	Combined faults are conserved, and restrictive Silver omits one received valid event.
Does a second implementation agree?	Exact source/aggregate comparisons agree for all 15 saved runs.
Can an energy result be interpreted safely?	Comparable support is explicit and unmatched periods retain unknown exposure.

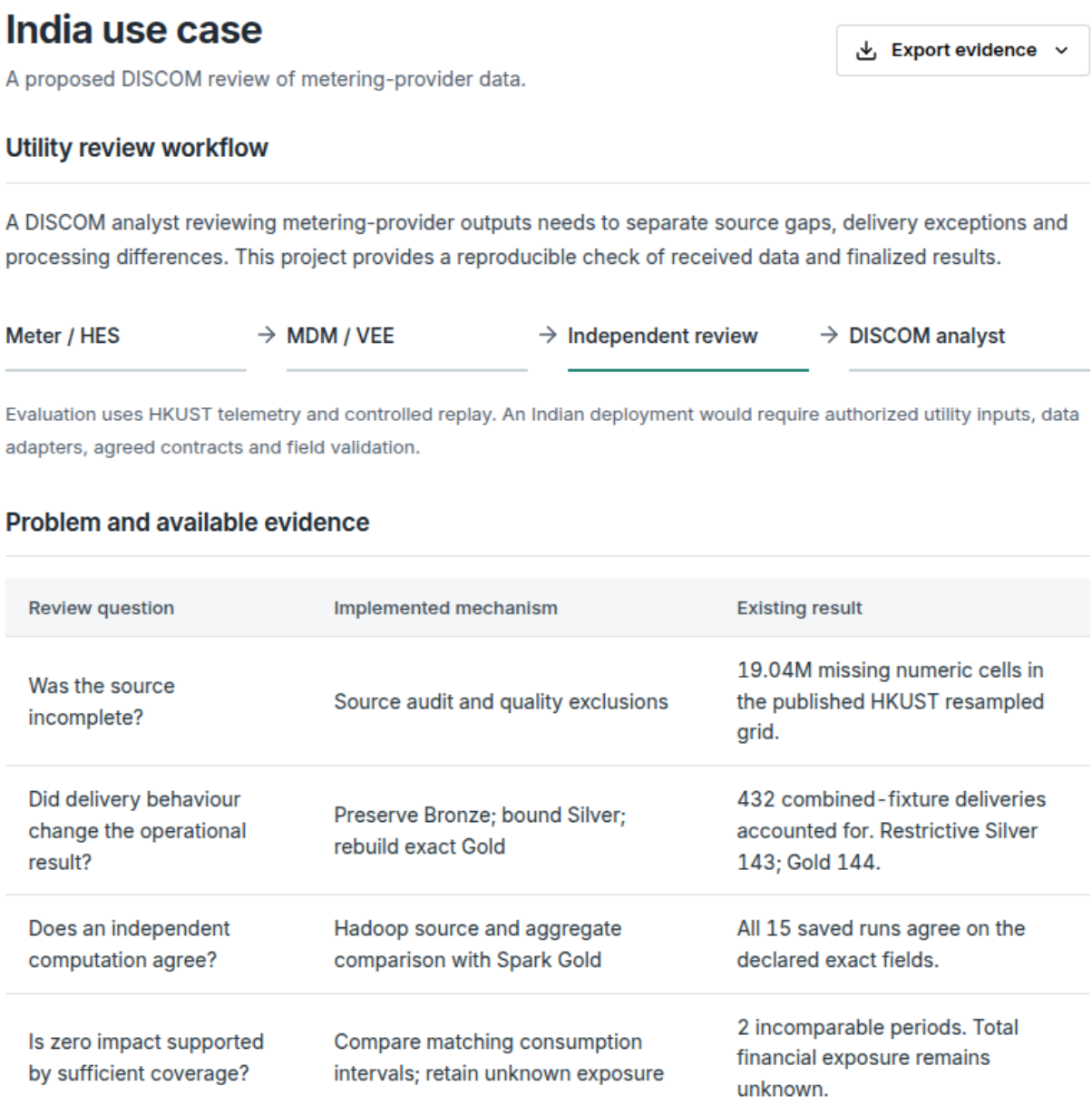


Figure 11: Proposed DISCOM review workflow and its connection to the implemented checks.

10.3 Software verification and handover

The submission checks cover parsing and precision, deterministic identity, replay contracts, conservation, finalization, independent computation and presentation bindings. The current non-distributed verification records 582 passing Python tests and seven skips, plus eight frontend tests. Desktop browser checks cover 74 page/viewport and interaction-state cases. No new distributed replay was needed to build this report.

Table 10: Acceptance evidence available in the submission.

Requirement	Verification basis
Account for received deliveries	Saved closure totals and recomputed conservation equalities.
Reconstruct exact source events	Gold/source comparisons and controlled-fault semantic output hashes.
Use an independent reference	Standalone Hadoop programs and repository independence scan.
Recover the same semantic answer	Four matching output pairs for the recovery fixture.
Reject unsupported publication	Contract tests, manifest bindings and tampered-evidence browser check.
Provide a usable desktop viewer	Frontend checks, browser interaction checks and 74 layout cases.

The seven Python skips comprise four Windows-specific rename checks, one inapplicable mutation fixture, one duplicate-sheet case that openpyxl cannot construct and one unavailable PySpark-dependent check. They are not counted as passes. The reported software checks supplement the completed experiment evidence.

The repository includes code, configurations, tests, committed summaries, report source, figures and a six-minute demonstration guide. The portable bundle includes the compiled report, standalone LaTeX assets, source snapshot and full dashboard captures. Raw workbooks and original large runtime directories are separate. The README documents both the lightweight viewer launch and the experiment environment. A demonstration can follow source audit, combined faults, independent verification, watermark coverage and recovery without a live large-scale replay.

11 Discussion and conclusion

11.1 What the project establishes

The implemented contribution is an evidence-preserving pipeline that combines deterministic delivery schedules, an operational stream, exact batch reconstruction, independent computation, recovery checks and a read-only evidence presentation. Its value is demonstrated by concrete distinctions: deliveries differ from source events, Silver differs from exact Gold, and computation agreement differs from source adequacy.

11.2 Evaluation limits

Several limits determine how the results can be used. The workloads are historical replay on one physical machine. Fault schedules are controlled rather than estimated from an Indian utility network. The scale selections are source-identifier prefixes rather than continuous meter-days. Canonicalization is shared by both engines. Absolute timestamps depend on an unconfirmed timezone assumption. Physical measurement accuracy, tariff correctness, live-system reliability and actual financial savings were not evaluated. The final scale evaluation ends at 1M selected source records.

The zero comparable-energy subtotal is a valid scoped result. It also reflects the chosen experiment and comparison definition: duplicate deliveries retain unchanged values, malformed deliveries accompany valid originals, and unmatched interval supports are not priced. A cumulative-register series can lose an intermediate observation without changing the energy difference between unchanged endpoints. The project therefore reports the observable accounting and coverage effects with their comparison coverage.

All 15 saved runs pass their declared source-record and aggregate comparisons. Exact Gold retains the valid source event omitted by restrictive Silver, and the interrupted fixture reproduces four semantic artifacts. These findings support the implemented recovery and independent-verification design through 1M selected source records. The proposed utility-review workflow requires further evaluation on authorized utility data before operational adoption.

11.3 Future work

Further work could evaluate realistic communication-loss histories, continuous-window consumption effects or cheaper selective verification. Such extensions would require new baselines and experiments. They are outside this submitted scope.

References

[1] M. Li, Z. Wang, Y. Qu, K. M. Chui and M. Leung-Shea, "A multi-year campus-level smart meter database," *Scientific Data*, vol. 11, article 1284, 2024. doi:10.1038/s41597-024-04106-1.

[2] HKUST smart-meter dataset, Dryad, deposited version 5, 1 August 2024. doi:10.5061/dryad.k3j9kd5h6. Registration and package details: repository `docs/dataset-audit.md`.

[3] Brick Schema, "Brick ontology and documentation." `https://brickschema.org/`. Accessed 13 November 2026.

[4] Apache Spark, "Structured Streaming programming guide," 3.5-series documentation. `https://spark.apache.org/docs/3.5.6/structured-streaming-programming-guide.html`. Consulted for watermark and deduplication semantics. Experiment image: Spark 3.5.1.

[5] Apache Kafka, "Design," version 3.7 documentation. `https://kafka.apache.org/37/design/design/`. Accessed 13 November 2026.

[6] Apache Hadoop, "Hadoop Streaming," version 3.4.3 documentation. `https://hadoop.apache.org/docs/r3.4.3/hadoop-streaming/HadoopStreaming.html`. Accessed 13 November 2026.

[7] Maharashtra State Electricity Distribution Co. Ltd., AMISP tender MMD/T-NSC-06/0922, 19 September 2022, section 2.4.4, PDF p. 185. Tender document.

[8] Oracle Utilities Analytics, "Performance," Meter Data Management metric reference. Quality, timeliness and estimation documentation. Accessed 13 November 2026.

[9] E. Garrison and J. New, "Quality control methods for advanced metering infrastructure data," *Smart Cities*, vol. 4, no. 1, pp. 195–203, 2021. doi:10.3390/smartcities4010012.

[10] S. Kumar and A. Subramaniam, "From Meters to Machine Learning: Building AI-Ready Utilities in India," CSIS, 30 October 2026. Original utility survey and analysis.

[11] Comptroller and Auditor General of India, Report No. 2 of 2025, Tripura, year ended 31 March 2024, chapter IV, printed p. 144. Historical smart-grid project observations.